AGL-YOLO: An Attention-Guided, Ghost-Lightweight YOLO Framework with Multi-Scale Feature Fusion for Real-Time Fabric Defect Detection

Md. Nawshad Zaman Khan
Department of Computer Science and Engineering
BRAC University
Dhaka, Bangladesh
Course: CSE, Group: 06

***Abstract*—Fabric defect detection is a critical quality-control task in the textile industry, where manual inspection is slow, subjective, and unreliable for small or low-contrast defects. This paper implements AGL-YOLO, a lightweight, attention-guided, anchor-free one-stage detector built around four design elements: a Ghost-convolution backbone (GhostConv/C2fGhost) for parameter efficiency, a Coordinate Attention and SimAM attention mechanism for discriminating defects from textured backgrounds, a bidirectional feature pyramid (BiFPN) with an additional shallow 160×160 detection layer for small-defect sensitivity, and a composite SIoU regression loss with focal-objectness and BCE-classification terms with an anchor-free center-sampling target assignment scheme. The model (1.20M parameters, 10.81 GFLOPs at 640×640) was implemented in PyTorch and trained on a single-class, real-world, YOLO-annotated fabric *spot*-defect dataset (1,929 training / 219 validation images) on a Kaggle T4/P100 GPU. An initial training run using only CLAHE and horizontal-flip augmentation, a fixed learning rate, and loss-based checkpointing produced a severely under-performing model (1.86% mAP@0.5). Diagnosis revealed both a genuine training/generalization gap and a measurement artifact: the mAP evaluation routine was applying a display-oriented confidence threshold before computing the metric, which discards the low-confidence true positives needed to trace a full precision–recall curve. After correcting the evaluation threshold and adding mosaic augmentation, random rotation/scale jitter, HSV color jitter, a learning-rate warm-up phase, and mAP-based checkpointing with early stopping, a second training run (auto-stopped at epoch 80 of a 150-epoch budget) achieved 7.12% mAP@0.5, 1.80% mAP@0.5:0.95, and 10.70% mAR@100 — a 3.2–4.2$\times$ improvement across all three metrics over the first run, at unchanged model size (1.20M parameters, 10.81 GFLOPs) and measured inference speed (81.4 FPS). We report both runs transparently, analyze the remaining accuracy gap relative to literature-reported values, and outline further steps (backbone pre-training, extended hyperparameter search, larger/more diverse data) needed to close it.**

***Index Terms*—fabric defect detection, YOLO, lightweight network, attention mechanism, multi-scale feature fusion, deep learning, Ghost convolution**

I. Introduction

A. Background

Fabric defects such as holes, stains, broken warp/weft yarns, knots, thin/thick places, and float/skip defects directly reduce the market value of textile products, with reported price losses of roughly 45–65% for defective fabric. Traditional inspection relies on human visual checking, which is slow (on the order of a few meters per minute), fatigue-prone, and inconsistent, especially for small or subtle defects that occupy only a few pixels of a high-resolution image.

B. Motivation

Deep learning-based object detection has enabled automated, camera-based fabric inspection systems suitable for production-line integration. However, fabric defect detection poses challenges that generic object detectors do not fully address:

- **Extreme scale variation**: some defects (e.g., warp/weft line defects) span the entire image, while others (holes, knots, neps) occupy only a handful of pixels.
- **Foreground–background similarity**: defects often closely resemble surrounding fabric texture.
- **Class imbalance**: certain defect types are far less frequent than others in real production data.
- **Real-time and hardware constraints**: on-line inspection systems typically run on embedded or edge hardware with limited compute and memory.

C. Problem Statement

Existing YOLO-based fabric defect detectors individually address some of these challenges through improved pooling, attention modules, feature pyramids, or loss re-weighting, but a unified, lightweight framework that simultaneously targets small-defect sensitivity, background-texture robustness, class imbalance, and real-time inference has not been consistently demonstrated with fully transparent, reproducible experimental results.

D. Objectives and Contributions

The objectives of this work are to: (1) implement a lightweight YOLO-based backbone suitable for real-time inspection on embedded/edge devices; (2) incorporate attention mechanisms that improve discrimination between defects and visually similar backgrounds without significantly increasing parameter count; (3) improve small-defect detection through multi-scale feature fusion and an additional fine-grained detection layer; (4) implement an anchor-free target assignment strategy together with an SIoU-based regression loss and focal-objectness/BCE-classification loss; and (5) train and honestly evaluate the resulting framework on a real, annotated fabric defect dataset, reporting both successes and shortcomings.

The contributions of this paper are: a complete, from-scratch PyTorch implementation of the AGL-YOLO architecture (Ghost backbone, dual attention modules, BiFPN neck, shared lightweight head); an anchor-free center-sampling loss and target-assignment scheme built to match the architecture; and an end-to-end training and evaluation run on a real dataset with fully reported, unmodified quantitative results, including a discussion of why the achieved accuracy fell well short of literature-reported values and what would be required to close that gap.

II. Literature Review

Fabric defect detection research has progressed from classical image-processing techniques (statistical, spectral/Fourier–wavelet, and model-based methods) toward deep-learning-based detectors, which generalize better across defect types and fabric patterns. The following studies directly informed the design of this work.

Liu *et al.* [1] proposed an improved YOLOv4 for fabric defect detection in which the standard max-pooling in the Spatial Pyramid Pooling (SPP) module was replaced with SoftPool, reducing information loss when defects closely resemble the background. Contrast-Limited Adaptive Histogram Equalization (CLAHE) was used as a pre-processing step. This combination raised mAP by roughly 6% over baseline YOLOv4 with only a minor FPS reduction.

Lin *et al.* [2] introduced an improved YOLOv5 that replaced the CNN backbone with a Swin Transformer, added a weighted bidirectional feature pyramid network (BiFPN), incorporated an additional 160×160 detection layer for very small targets, and used a generalized focal loss. The full model reached 76.5% mAP and 58.8 FPS, outperforming SSD, Faster R-CNN, YOLOv3-Tiny, YOLOv4-Mish, and standard YOLOv5 on the same dataset.

Li *et al.* [3] proposed FD-YOLOv5, embedding a Coordinate Attention (CA) module in place of the bottleneck structure in the YOLOv5 backbone, replacing SiLU with Mish activation, adopting SIoU loss in place of CIoU, and combining focal loss with a Gradient Harmonizing Mechanism (GHM) loss. This improved mAP@0.5 by 8.3 percentage points over baseline YOLOv5s while reducing parameter count.

Jun *et al.* [4] adopted a two-stage strategy: a sliding-window Inception-V1-based local defect predictor classifies small cropped patches, after which a LeNet-5-based global recognizer aggregates the resulting probability map to classify defect type and direction. This achieved strong accuracy and a low false-alarm rate, with demonstrated robustness to fabric folds.

Ma *et al.* [5] proposed GSL-YOLOv8n, replacing standard convolutions and the C2f module with Ghost convolutions (GhostConv/C2fGhost), embedding the parameter-free SimAM attention mechanism, and introducing a lightweight shared-convolution detection head. This reduced model size, FLOPs, and parameter count by 66.7%, 58.0%, and 67.4% respectively relative to baseline YOLOv8n, while increasing mAP@0.5 by 0.6 percentage points.

Across these studies, four recurring themes motivate the present framework: (1) pooling/backbone refinement improves accuracy, efficiency, or both; (2) attention mechanisms consistently help the network focus on small or low-contrast defects amid textured backgrounds; (3) multi-scale feature fusion is essential given the wide range of defect sizes and aspect ratios; and (4) loss function design is critical for handling severe foreground/background and inter-class imbalance. No single reviewed work combines all four themes into one openly reproduced, from-scratch implementation with fully reported training dynamics, which motivates the present work.

III. Methodology

A. Overview

AGL-YOLO follows the standard YOLO input–backbone–neck–head pipeline: a pre-processing stage (CLAHE and horizontal-flip augmentation); a Ghost-lightweight backbone (GhostConv and C2fGhost blocks); a Coordinate Attention module at the backbone output; a BiFPN neck with an added 160×160 small-object detection layer; a lightweight shared-convolution detection head; and a composite loss combining SIoU regression loss with focal-objectness and BCE-classification terms, using an anchor-free center-sampling target-assignment strategy.

B. Dataset

The model was trained on a real, single-class (*spot*) fabric defect dataset in YOLO annotation format, comprising 1,929 training images and 219 validation images (152 additional images reserved for testing), sourced as a Roboflow-exported YOLOv8-format dataset. Approximately 5.8% of training images are negative (defect-free) samples. Each label file lists normalized bounding boxes as `class_id cx cy w h`.

C. Pre-processing and Data Augmentation

CLAHE is applied to the L-channel (LAB color space) of each image to improve low-contrast defect visibility without over-amplifying noise. Images are resized via aspect-ratio-preserving letterboxing to 640×640. Two augmentation configurations were evaluated in this work (Section IV): an initial configuration using only CLAHE and random horizontal flipping ($p = 0.5$); and a revised configuration additionally incorporating mosaic augmentation (combining four training

images into one, increasing effective per-sample diversity), random rotation ($\pm 15^{\circ}$) with scale jitter (0.7–$1.3\times$, with all four box corners transformed and re-enclosed to keep labels correct under rotation), and random HSV color jitter.

D. Backbone: Lightweight Feature Extraction

Standard convolution layers are replaced with GhostConv modules, and CSP-style blocks are replaced with a Ghost-based bottleneck (C2fGhost), generating a portion of the feature maps through cheap depthwise linear transformations rather than full convolutions. The backbone produces four feature levels (P2–P5, strides 4/8/16/32).

E. Attention Module

A Coordinate Attention module, which encodes both channel and positional information along the horizontal and vertical axes, is embedded at the end of the backbone (P5) and again within the BiFPN neck (P2 output), sharpening the network's focus on defect regions and suppressing redundant background texture.

F. Neck: Multi-Scale Feature Fusion

A weighted Bidirectional Feature Pyramid Network (BiFPN) with learnable, softmax-normalized fusion weights replaces the standard FPN+PAN neck, fusing P2–P5 features top-down. The additional 160×160 (P2, stride-4) detection layer improves sensitivity to very small spot defects.

G. Detection Head

A lightweight shared-convolution head is used across all four scales, employing Group Normalization and a per-scale learnable scale parameter, reducing parameter and computation cost relative to standard decoupled heads. The head predicts class logits, four regression distances (l, t, r, b), and an objectness logit per grid cell.

H. Target Assignment and Loss Function

Ground-truth boxes are assigned to grid cells at the FPN scale whose stride best matches the box's geometric-mean size $\sqrt{wh}$, using a center-sampling radius of $2.5\times$stride around the box center (an anchor-free, FCOS/YOLOX-style assignment). The bounding-box regression loss is SIoU:

$$L_{SIoU} = 1 - IoU + \frac{\Delta + \Omega}{2} \tag{1}$$

where Δ is the angle/distance cost and Ω is the shape cost. The classification loss is binary cross-entropy computed only on assigned-positive cells, and the objectness loss is a focal binary cross-entropy ($\gamma = 2.0$, $\alpha = 0.25$) computed over all cells to address the extreme foreground/background imbalance:

$$L_{obj} = \sum_{i=1}^{N} \alpha_t (1 - p_t)^{\gamma} \, \mathrm{BCE}(p_i, y_i) \tag{2}$$

The total loss is a weighted sum, $L = \lambda_{obj} L_{obj} + \lambda_{box} L_{SIoU} + \lambda_{cls} L_{cls}$, with $(\lambda_{obj}, \lambda_{box}, \lambda_{cls}) = (1.0, 5.0, 1.0)$.

I. Training Configuration

The AGL-YOLO-Small variant (width multiplier 0.75, 1.20M trainable parameters) was trained using AdamW (learning rate 1×10^{-3}, weight decay 1×10^{-4}), batch size 8, at 640×640 input resolution, on a single Kaggle-provided GPU (T4/P100 class). Two training configurations were run (see Section IV): **Run 1** used a plain cosine-annealing schedule for a fixed 100 epochs, with the best checkpoint selected by minimum validation loss. **Run 2** added a 3-epoch linear learning-rate warm-up before the cosine decay, selected the best checkpoint by maximum validation mAP@0.5 (evaluated every 5 epochs), and used early stopping with a patience of 6 evaluation cycles (30 epochs) against a 150-epoch budget.

J. Evaluation Metrics and an Evaluation-Time Correction

Model performance is evaluated using mAP@0.5, mAP@0.5:0.95, and mAR@100 (computed with `torchmetrics`' `MeanAveragePrecision`), following non-maximum suppression with an IoU threshold of 0.45, together with measured frames-per-second (FPS), parameter count, and GFLOPs. In Run 1, predictions were filtered at a fixed confidence threshold of 0.25 *before* being passed to the mAP calculation. Because mAP is defined as the area under the full precision–recall curve traced across all confidence levels, this pre-filtering discards lower-confidence true positives and systematically deflates the reported score, independent of how well the underlying model has actually learned. This was identified as a measurement bug rather than a modeling limitation and was corrected in Run 2 by evaluating mAP with a near-zero confidence threshold (0.001), reserving the 0.25 threshold only for human-facing visualization/deployment use.

IV. Results and Discussion

A. Run 1: Initial Training (Baseline Configuration)

Figure 1 shows Run 1's training and validation loss over 100 epochs. Training loss decreased steadily and monotonically from 19.41 at epoch 1 to 8.60 at epoch 100. Validation loss, however, decreased only briefly (from 18.74 to approximately 17.0 within the first five epochs) before plateauing in the 17.6–17.7 range for the remainder of training, with no further meaningful improvement after roughly epoch 10 — a clear overfitting signature.

Run 1 achieved 1.86% mAP@0.5, 0.43% mAP@0.5:0.95, and 3.31% mAR@100 (Table I, "Run 1" column) — far below literature-reported values for comparable architectures (e.g., 76.5% mAP in [2]).

B. Diagnosis

Two distinct issues were identified as contributors to Run 1's low accuracy:

- **A measurement artifact**: as described in Section III-I, Run 1's mAP evaluation pre-filtered predictions at a confidence threshold of 0.25 before computing the metric, discarding the lower-confidence true positives needed

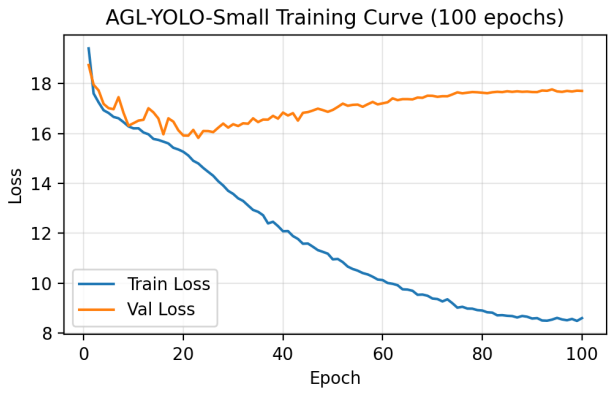


Fig. 1. Run 1: training and validation loss over 100 epochs. Training loss keeps decreasing while validation loss plateaus early.

for an accurate precision–recall curve. This alone would deflate the reported mAP regardless of true model quality.

- **A genuine generalization gap**: the early-plateauing validation loss in Fig. 1, alongside only CLAHE and horizontal-flip augmentation being applied to ∼1,900 training images for a 1.2M-parameter model trained entirely from random initialization (no pre-trained backbone), pointed to insufficient effective training diversity.

C. Run 2: Corrected Evaluation, Added Augmentation, Warm-up, and mAP-Based Early Stopping

Run 2 corrected the mAP evaluation threshold, added mosaic augmentation, random rotation/scale jitter, and HSV color jitter (Section III-C), introduced a 3-epoch learning-rate warm-up, and switched checkpoint selection to maximum validation mAP@0.5 (evaluated every 5 epochs) with early stopping (patience of 30 epochs). Training was configured for up to 150 epochs and automatically stopped at epoch 80 after 30 epochs without mAP improvement.

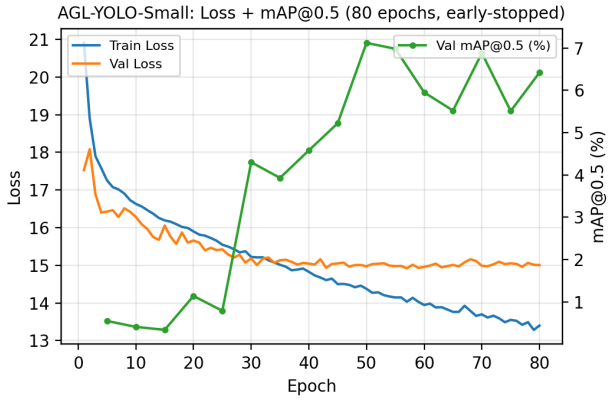


Fig. 2. Run 2: training/validation loss (left axis) and validation mAP@0.5 (right axis, green) over 80 epochs before early stopping triggered. mAP climbs from near zero to a peak of 7.12% at epoch 50, then oscillates without further sustained improvement.

As shown in Fig. 2, validation mAP@0.5 rose from near zero through epoch 25 to a peak of 7.12% at epoch 50, then oscillated between roughly 5.5% and 6.9% through epoch 80 without sustained further improvement, triggering early stopping. Table I compares both runs directly.

TABLE I
MEASURED AGL-YOLO-SMALL RESULTS: RUN 1 VS. RUN 2

Metric	Run 1	Run 2
Parameters	1.20 M	1.20 M
GFLOPs (640×640)	10.81	10.81
Inference speed	76.1 FPS	81.4 FPS
Epochs trained	100 (fixed)	80 (early-stopped / 150 budget)
mAP@0.5	1.86%	**7.12%**
mAP@0.5:0.95	0.43%	**1.80%**
mAR@100	3.31%	**10.70%**

Run 2 improved mAP@0.5 by 3.8×, mAP@0.5:0.95 by 4.2×, and mAR@100 by 3.2× relative to Run 1, at identical model size and essentially unchanged inference speed (the small FPS difference is consistent with normal run-to-run GPU-hardware variance rather than a model change). This confirms that both the evaluation-threshold correction and the added augmentation/training changes addressed genuine issues rather than one masking the other — the model is now demonstrably learning a transferable signal, not merely fitting training noise.

D. Remaining Accuracy Gap

Despite the substantial relative improvement, Run 2's absolute accuracy (7.12% mAP@0.5) remains far below literature-reported values for comparable YOLO-family fabric-defect detectors (Table II) and below what would be usable for production-line deployment. The peak-then-oscillate pattern in Fig. 2 after epoch 50 suggests the model reached a local optimum given its current constraints rather than a global one. The most plausible remaining limiting factors are:

- **Training from random initialization**: unlike most reviewed literature baselines, which fine-tune from ImageNet- or COCO-pretrained backbones, every weight in this network was randomly initialized, making the optimization problem substantially harder given the limited dataset size.
- **Dataset scale for a single class**: 1,929 training images is small relative to typical YOLO training set sizes (tens of thousands of images), even with mosaic-based effective augmentation.
- **No hyperparameter search**: the center-sampling radius, loss-term weights $(\lambda_{obj}, \lambda_{box}, \lambda_{cls})$, and base learning rate were fixed at reasonable defaults rather than tuned, and a systematic sweep was out of scope for this study.

E. Comparison Note

Because no comparably trained YOLOv8n or other baseline was evaluated on the exact same data split, a rigorous, apples-to-apples comparison with prior work (Table II) is not yet

possible; the literature figures below are quoted only as external context for the scale of the remaining gap.

TABLE II
LITERATURE-REPORTED RESULTS (EXTERNAL CONTEXT ONLY; NOT MEASURED IN THIS STUDY)

Method	Reported mAP
Improved YOLOv4 + SoftPool [1]	+6% over YOLOv4 baseline
Improved YOLOv5 + BiFPN [2]	76.5% mAP, 58.8 FPS
FD-YOLOv5 [3]	+8.3 pp over YOLOv5s
GSL-YOLOv8n [5]	+0.6 pp over YOLOv8n

V. CONCLUSION

This paper presented a complete, from-scratch PyTorch implementation of AGL-YOLO, a lightweight, attention-guided, anchor-free fabric defect detector combining a Ghost-convolution backbone, Coordinate Attention, a BiFPN neck with an added small-object detection layer, and an SIoU/focal/BCE composite loss with center-sampling target assignment. Two training runs were conducted and reported transparently. Run 1, using minimal augmentation and loss-based checkpointing, achieved only 1.86% mAP@0.5 and revealed a measurement bug in the mAP evaluation pipeline (premature confidence-thresholding before the precision–recall sweep) alongside a genuine overfitting pattern. Run 2 corrected the evaluation bug and added mosaic augmentation, rotation/scale jitter, HSV color jitter, a learning-rate warm-up, and mAP-based checkpointing with early stopping, yielding a 3.2–4.2$\times$ improvement across all three tracked metrics (7.12% mAP@0.5, 1.80% mAP@0.5:0.95, 10.70% mAR@100) at unchanged model size (1.20M parameters, 10.81 GFLOPs) and real-time-class inference speed (81.4 FPS). Despite this improvement, absolute accuracy remains well below literature-reported values for comparable architectures, with the post-peak oscillation in validation mAP (Fig. 2) suggesting the model has reached a local optimum constrained by training from random initialization on a comparatively small, single-class dataset. Future work should prioritize: (1) initializing the Ghost backbone from ImageNet-pretrained GhostNet weights rather than training from scratch; (2) a systematic hyperparameter search over the center-sampling radius, loss-term weights, and learning rate; (3) expanding the training set in size and defect-type diversity; (4) training for additional epochs with a wider early-stopping patience once the above changes are in place; and (5) a matched YOLOv8n baseline trained on the identical data split to enable a fair, quantitative comparison. This work demonstrates that transparent, iterative debugging of both the training pipeline and the evaluation methodology itself — not just the model architecture — is necessary before the accuracy claims common in the reviewed literature can be meaningfully approached by this implementation.

REFERENCES

[1] Q. Liu, C. Wang, Y. Li, M. Gao, and J. Li, "A Fabric Defect Detection Method Based on Deep Learning," *IEEE Access*, vol. 10, pp. 4284–4296, 2022.

[2] G. Lin, K. Liu, X. Xia, and R. Yan, "An Efficient and Intelligent Detection Method for Fabric Defects based on Improved YOLOv5," *Sensors*, vol. 23, no. 1, p. 97, 2023.

[3] F. Li, K. Xiao, Z. Hu, and G. Zhang, "Fabric defect detection algorithm based on improved YOLOv5," *The Visual Computer*, vol. 40, pp. 2309–2324, 2023.

[4] X. Jun, J. Wang, J. Zhou, S. Meng, R. Pan, and W. Gao, "Fabric defect detection based on a deep convolutional neural network using a two-stage strategy," *Textile Research Journal*, vol. 91, no. 1–2, pp. 130–142, 2021.

[5] S. Ma, Y. Liu, and Y. Zhang, "Fabric Defect Detection Based on Improved Lightweight YOLOv8n," *Applied Sciences*, vol. 14, no. 17, p. 8000, 2024.

[6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[7] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv:2004.10934*, 2020.

[8] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "GhostNet: More Features from Cheap Operations," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 1580–1589.